DRAM ARCHITECTURE AND MEMORY CONTROLLERS

DRAM Architecture and Memory Controllers

Topics:

- Why DRAM matters
- DRAM cell organization
- Banks, rows and columns
- DRAM timing parameters
- Row buffer locality
- DRAM commands
- Memory controllers
- Scheduling policies
- Bandwidth vs latency
- Modern DRAM technologies
- DDR, LPDDR, HBM

Motivation

Modern processors are often limited by:

- Memory latency
- Memory bandwidth
- Memory contention

Even with large caches many accesses eventually reach DRAM

Thus DRAM architecture strongly affects system performance

Memory Hierarchy Recap

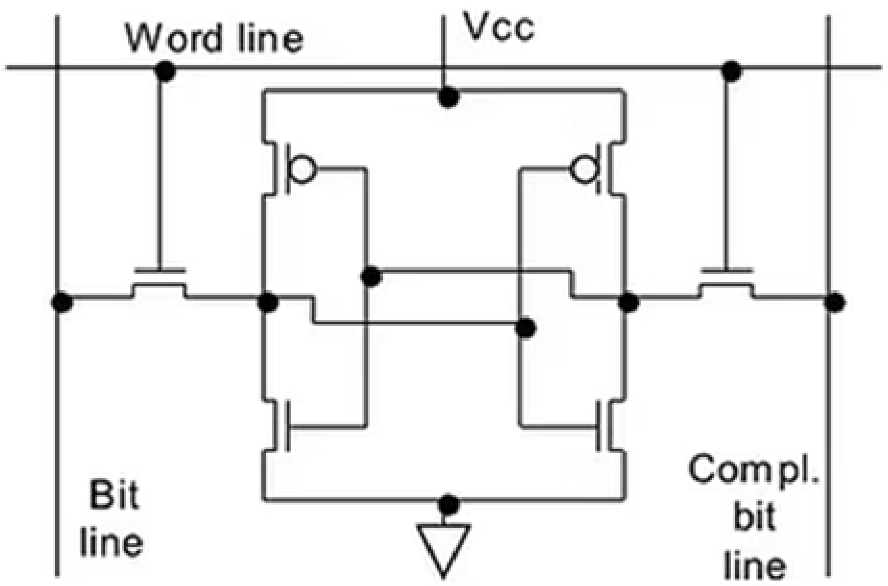
Level	Typical latency
Registers	~1 cycle
L1 cache	~1-4 cycles
L2 cache	~10 cycles
L3 cache	~30-50 cycles
DRAM	~100-300 cycles

DRAM is order of magnitude slower than caches!

DRAM vs SRAM

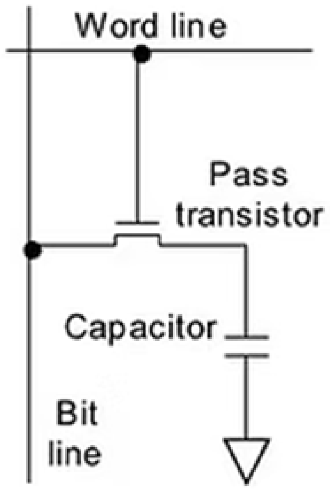
SRAM (used in cache)

- Fast
- Expensive
- Low density
- Uses flip-flops



DRAM

- Slower
- Very dense
- Cheap per bit
- Uses capacitors



DRAM Cell

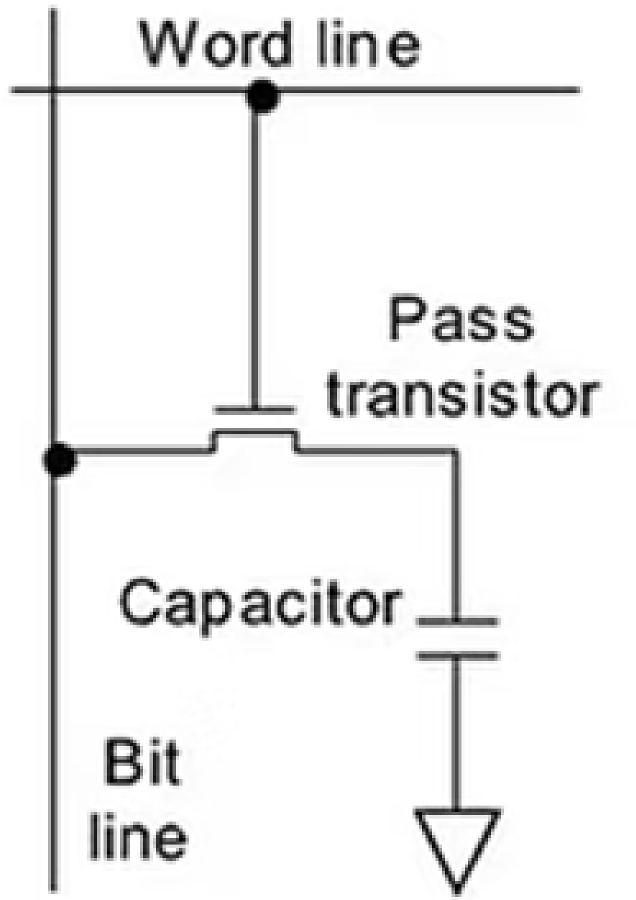
DRAM Storage

Each DRAM cell contains

- One transistor
- One capacitor

The capacitor stores charge representing

- 1 = charge
- 0 = discharged

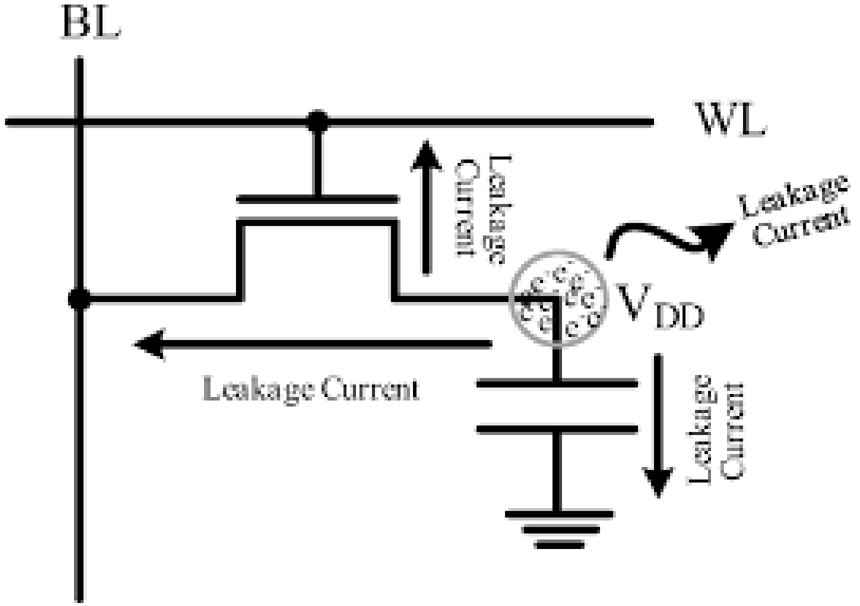


Problem: Charge Leakage

DRAM capacitors lose charge over time

Therefore data must be periodically refreshed

Typical refresh interval: 64 ms



Consequence of Refresh

Refresh operations:

- Consume bandwidth
- Increase latency
- Temporarily block accesses

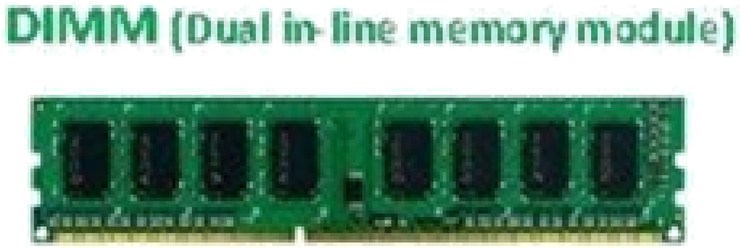
Refresh becomes a majority issue in

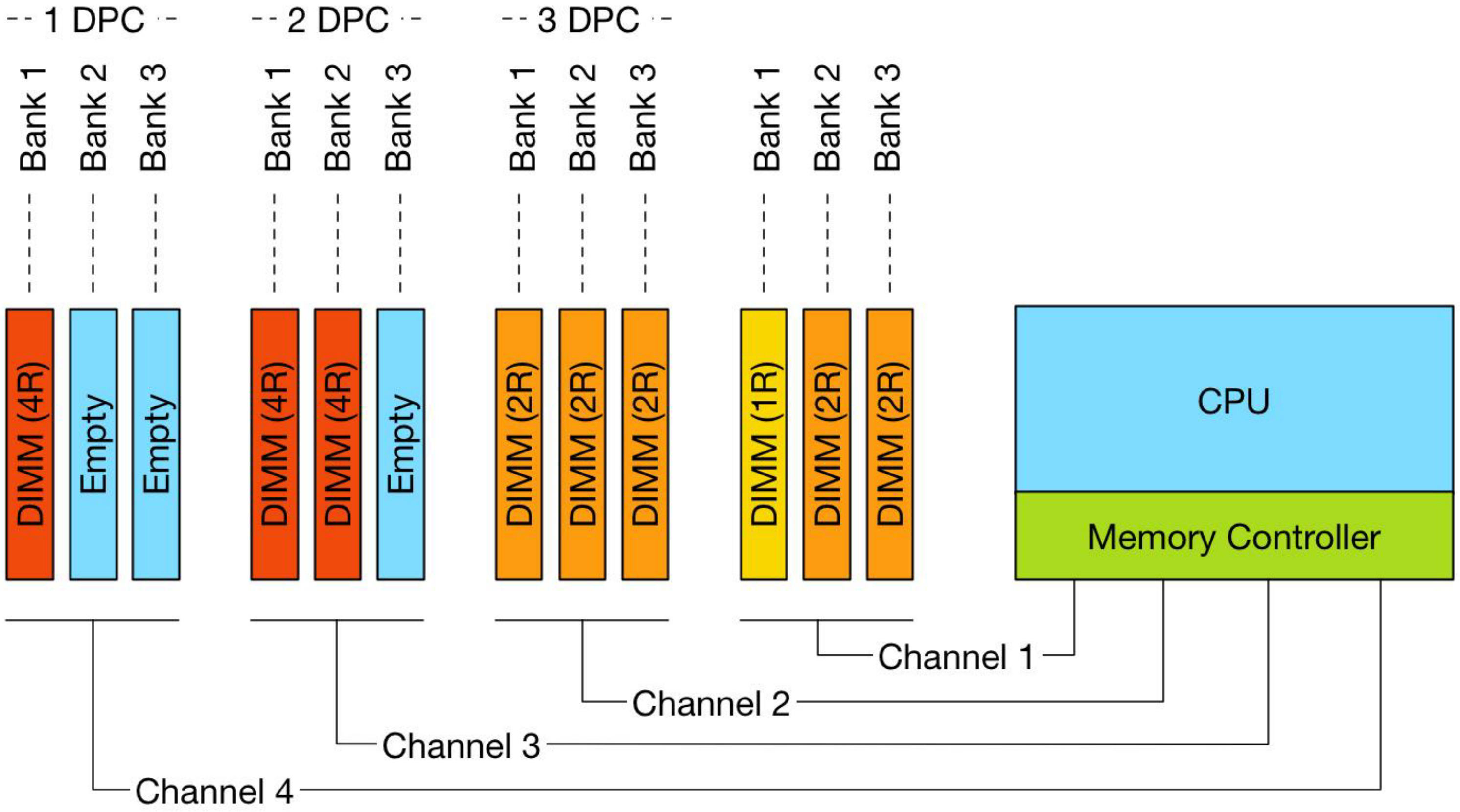
- Large memories
- High-density DRAM

DRAM Organization

DRAM is hierarchical
Structure:

- Channel
- DIMM
- Rank
- Chip
- Bank
- Row
- Column





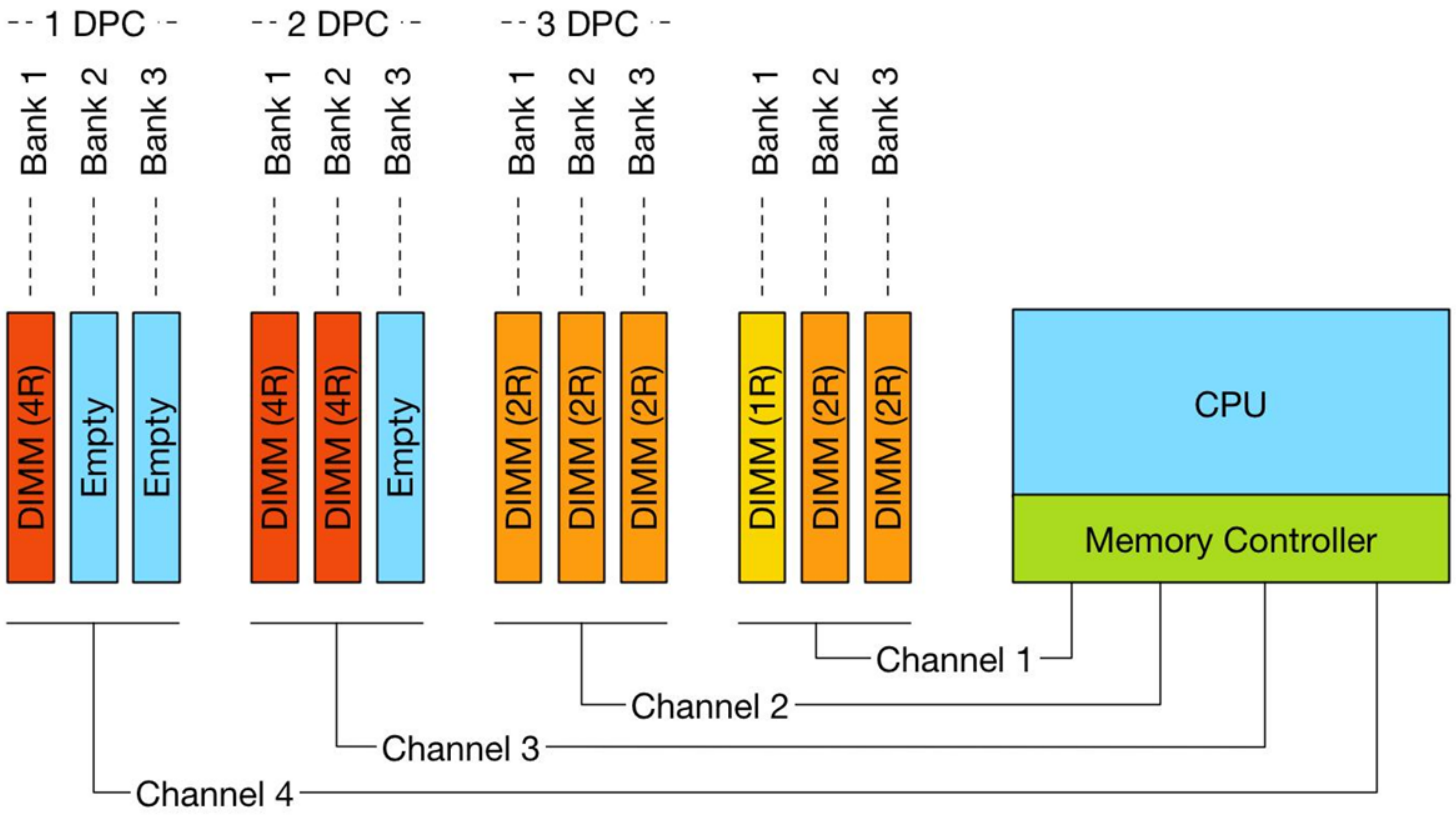
Channels

A channel is an independent communication path between:

- Memory controller
- DRAM modules

More channels:

- Higher bandwidth
- Parallel accesses



DIMMs and Ranks

DIMM is a physical memory module

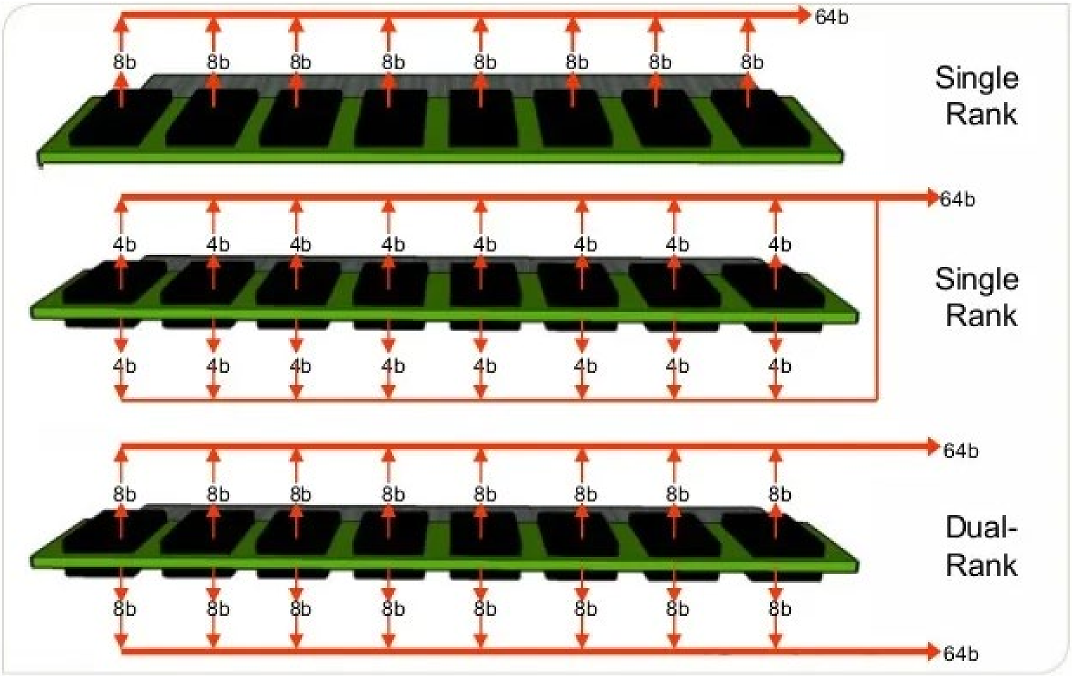
Rank is a group of DRAM chips operating together

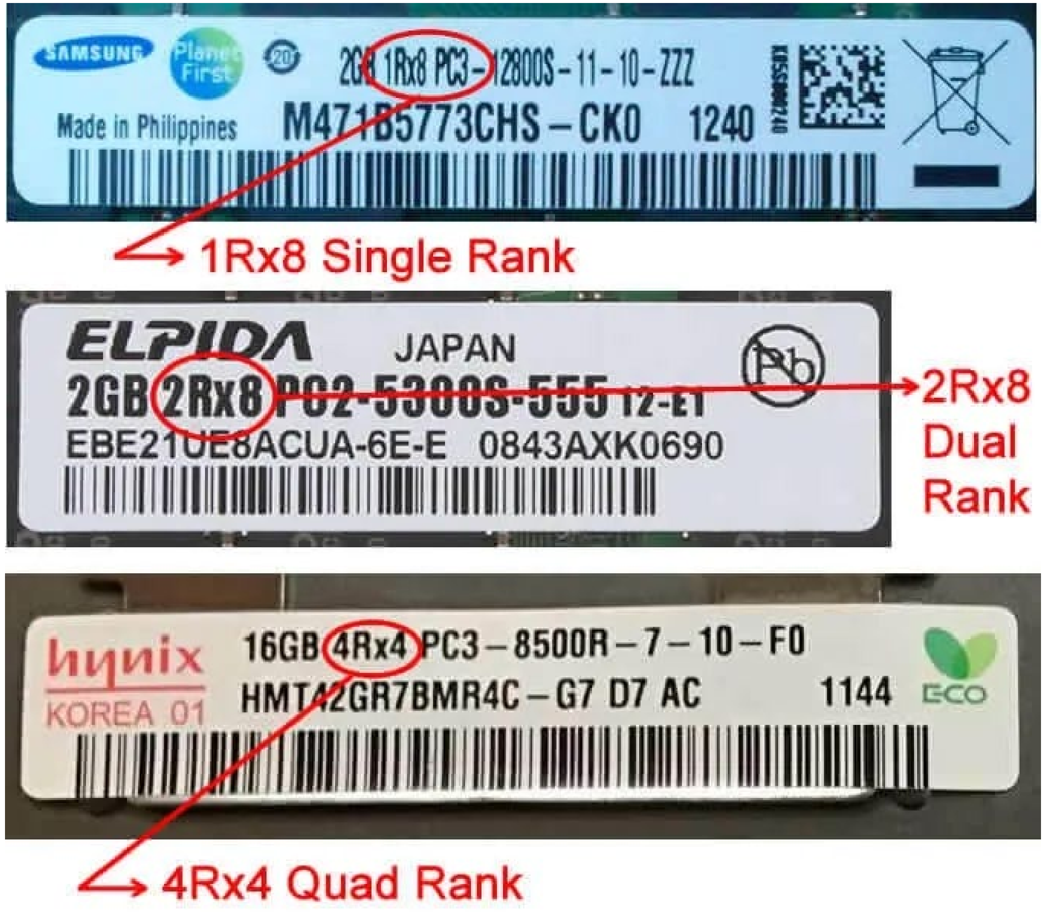
Ranks share:

- Command bus
- Address bus

But have separate chip-select signals

DIMMs and Ranks





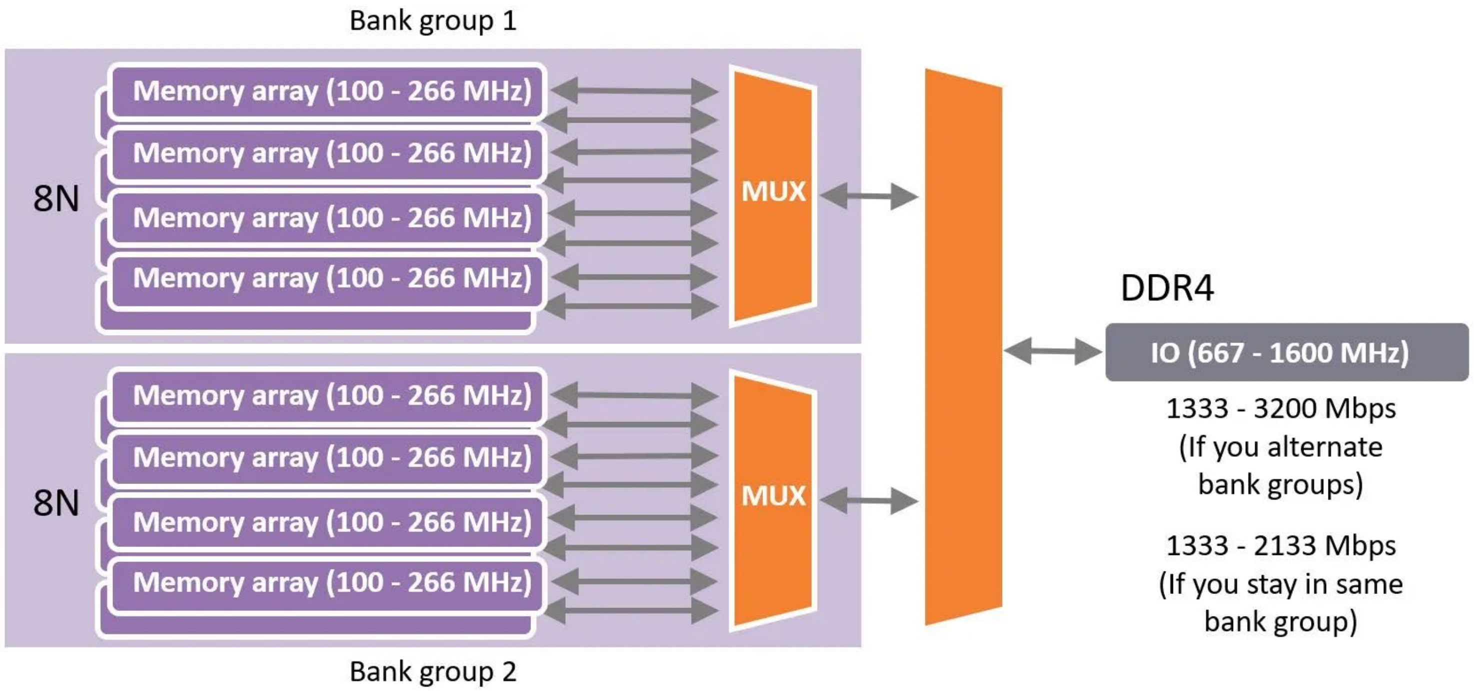
Banks

Each bank behaves approximatively like an independent memory array

Modern DRAM may contain:

- 8 banks
- 16 banks
- Bank groups

Banks allow parallelism



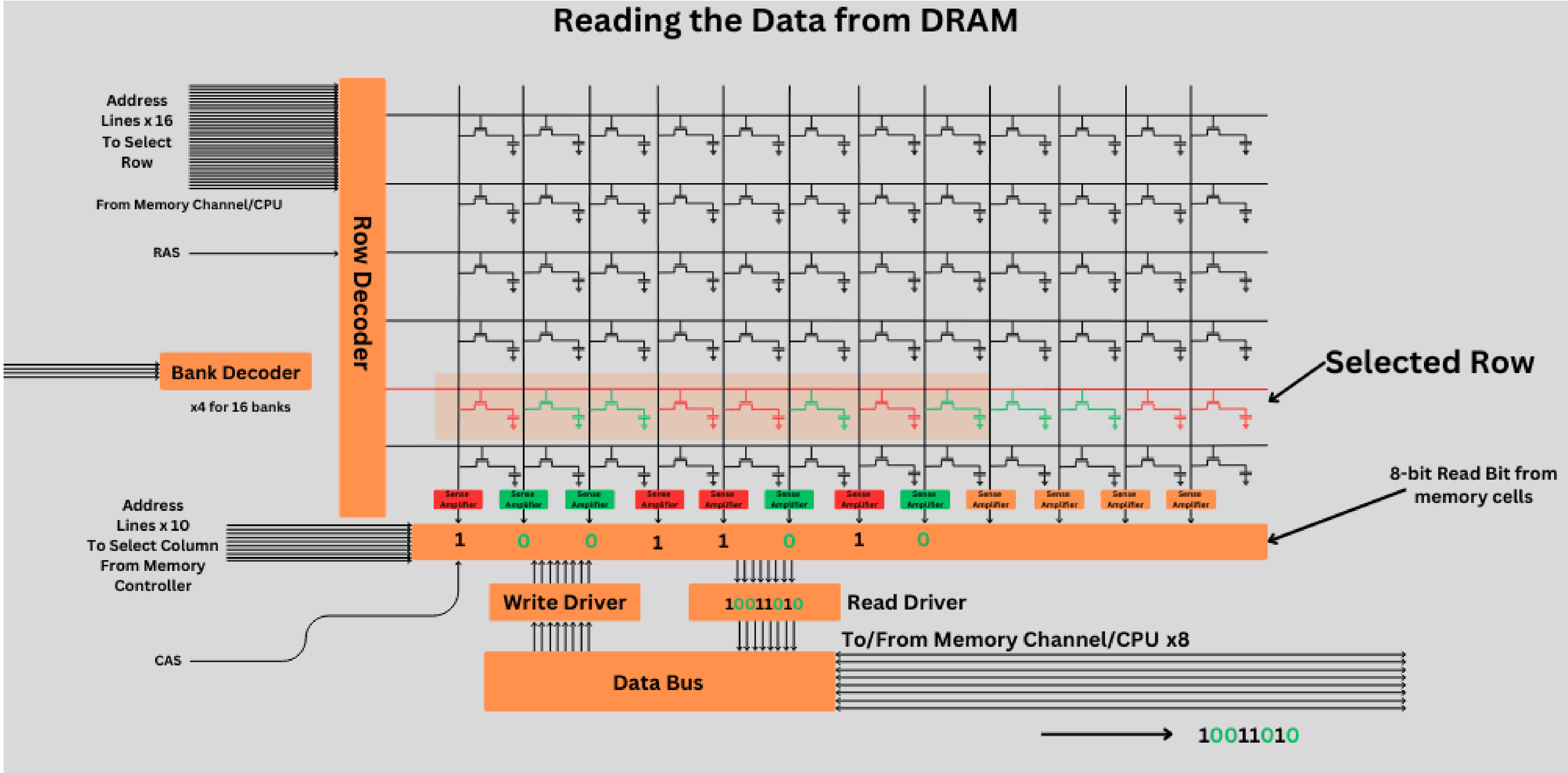
Rows and Columns

Inside each bank data are organized in rows and columns

Typical row size is several KB

It is important that the entire row is activated at once

Rows and Columns



Row buffer

Hey concept -> when a row is activated it is copied into the row buffer

The row buffer behaves similarly to a cache

If next access targets same row we have a row hit and a fast access

If access targets different row, the current row is closed and a new row is activated

Higher latency

DRAM Commands

Main DRAM commands:

- ACTIVATE -> opens a row
  - Entire row loaded into row buffer. Costly operation
- READ -> selects columns from row buffer
  - Access is relatively fast because row already active
- WRITE -> writes data into DRAM
- PRECHARGE -> before operating another row, the current row must be closed
- REFRESH -> refresh the DRAM cells

DRAM Access Sequence

Typical access sequence:

PRECHARGE
ACTIVATE
READ/WRITE
PRECHARGE

Most latency comes from row management

DRAM Timing Parameters

Important timings:

- t_{RCD} -> row to column delay
  - Time between ACTIVATE and READ/WRITE
- t_{CAS} -> column access strobe latency
  - Time between READ command and data availability
- t_{RP} -> row precharge time
  - Time required to close a row before opening another
- t_{RAS} -> row active time
  - Minimum time a row must remain active

These define minimum delays between commands

Full Timing Example

Example timing

ACTIVATE -> wait t_{RCD}
READ -> wait t_{CAS}
PRECHARGE -> wait t_{RP}

Total latency accumulates quickly

Row Buffer Hit vs Miss

Row Buffer Hit

Same row already open

Fast

Row Buffer Miss

Different row needed

Must:

- precharge
- activate new row

Much slower

Row Buffer Conflict

Worst case -> repeated accesses alternate between rows

Example:

Row A -> Row B -> Row A -> Row B

Causes repeated:

- Precharge
- Activate

Large latency increase

Memory Controllers

Hardware unit managing DRAM accesses

Responsibilities

- Scheduling
- Command generation
- Refresh handling
- Arbitration

Evolution of Memory Controllers

Older systems:

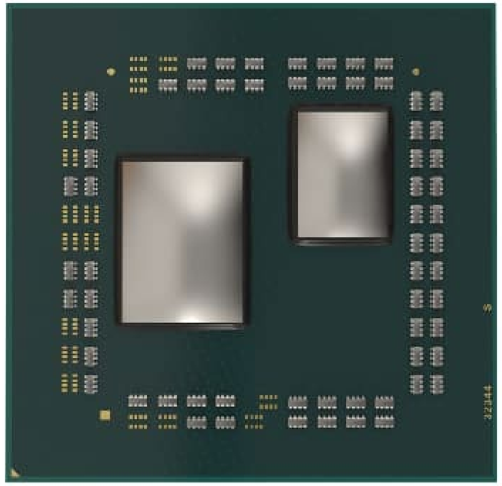
- Controller on chipset

Modern CPUs:

- Integrated memory controller (IMC)

Benefits:

- Low latency
- Higher bandwidth



Memory Controller Queues and Scheduling Problem

Request arrive from:

- Cores
- DMA
- I/O devices

Controller maintains request queues

Controller must decide which request to serve next

Objectives:

- Maximize throughput
- Minimize latency
- Preserve fairness

FCFS Scheduling

First-Come First Served

Serve oldest request first

Advantages:

- Simple
- Fair

Disadvantages

- Ignores row locality

FR-FCFS Scheduling

First Ready First-Come First Served

Prioritize:
1. Row buffer hits
2. Oldest requests

Widely used

Works because row hits avoid activate and precharge
Higher bandwidth and lower average latency

Downside of FR-FCFS Scheduling

May starve some request

Especially requests targeting closed rows

Trade-off:

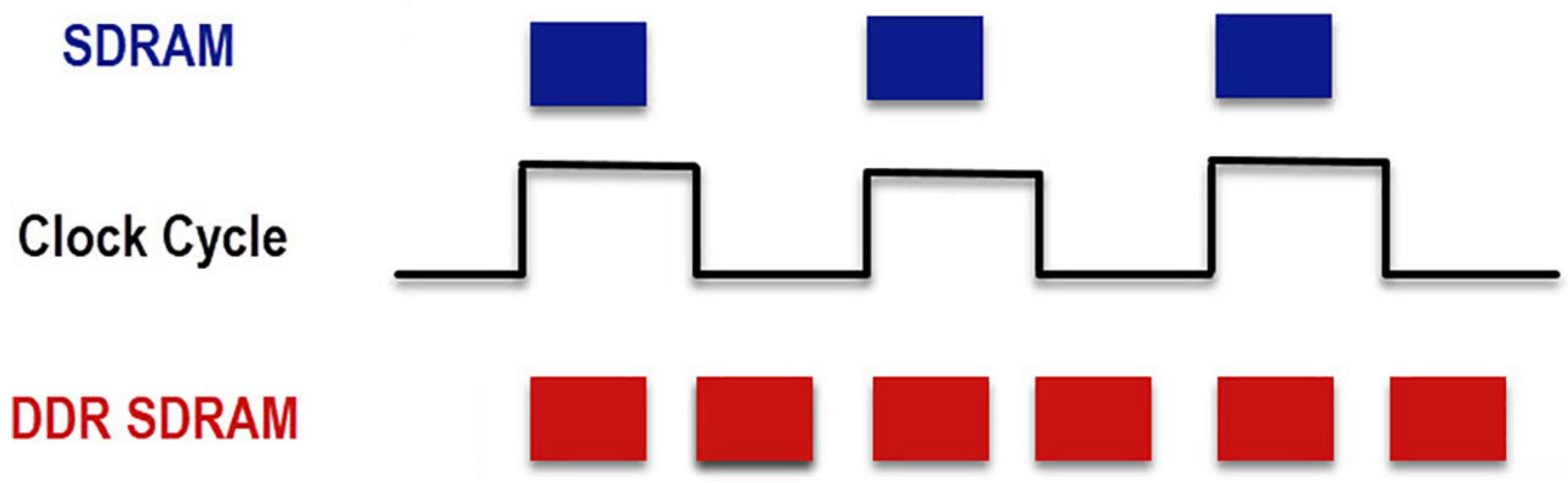
- Performance
- Fairness

DDR SRAM

Double Data Rate

Transfers data on rising edge and falling edge

Effectively doubles bandwidth



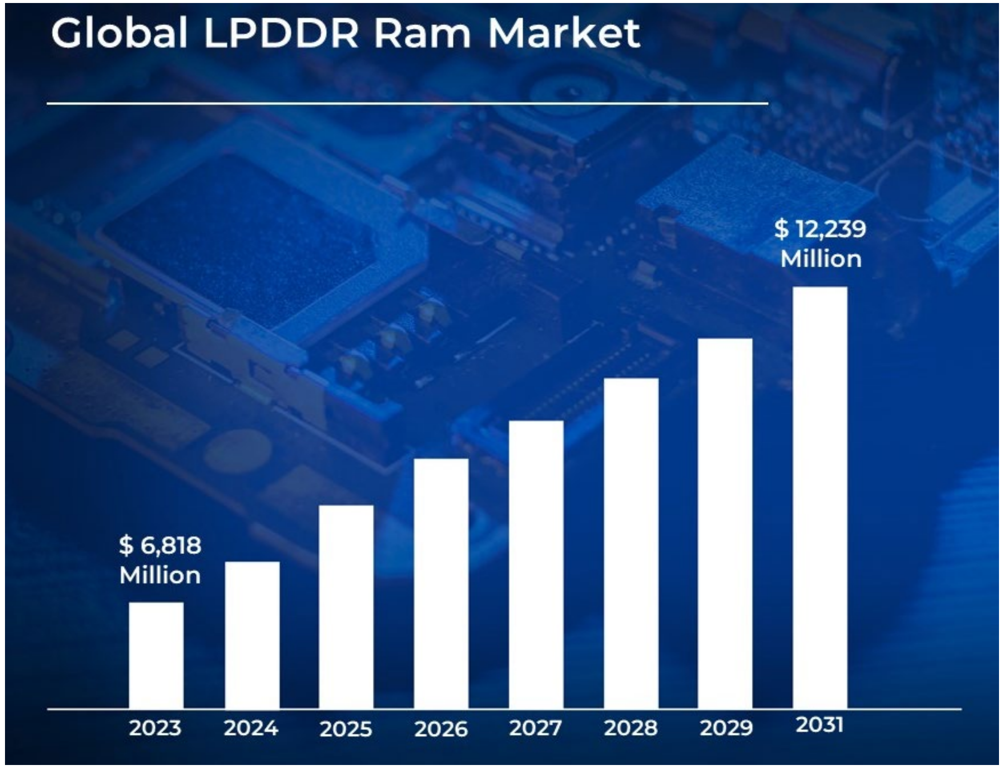
DDR Generations

Technology	**Bandwidth Increase**
DDR	Baseline
DDR2	Higher frequency
DDR3	Improved efficiency
DDR4	Bank groups
DDR5	More parallelism

Low Power DDR

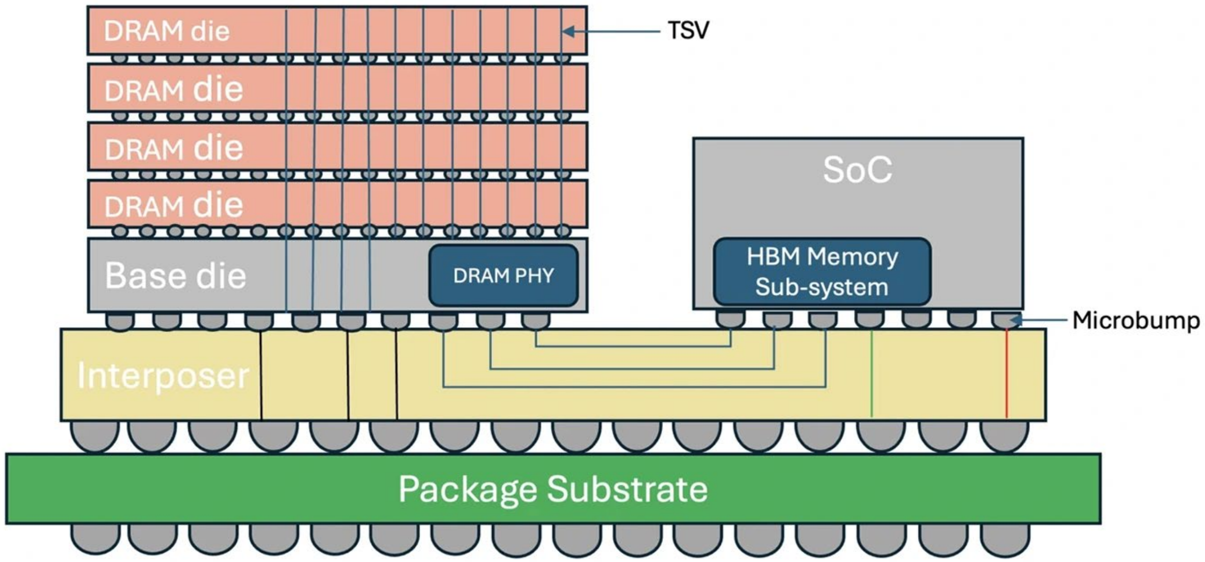
LPDDR are designed for mobile devices and energy efficiency

Optimizations are lower voltage and deep power states



High Bandwidth Memory

HBM are made up by 3D-stacked DRAM
Uses wide interfaces and TSVs (Through-Silicon Vias)
Provides enormous bandwidth and low energy per bit
Used in GPUs, AI accelerators



Closing

Key concepts:

- DRAM organization
- Banks and row buffers
- Timing parameters
- Memory controllers
- Scheduling policies
- DDR technologies
- HBM